🎬 AI Job Recommender App — Video Coding Guide

> **Recreate the Full Project from Scratch using Any AI IDE (Cursor / Antigravity / VS Code + Copilot)**

> ***How to use this guide*** *Each section has 4 parts: 1.* ***What this prompt does*** *— plain English, 2-3 lines 2.* ***Before running*** *— manual steps you must do first (accounts, keys, etc.) 3. 📋* ***The Prompt*** *— paste this exactly into your AI IDE 4. ✅* ***Checkpoint Prompt*** *— paste this into your AI IDE to verify everything works before moving on*

📋 Section 0 — Prerequisites Checklist

What this covers: Everything you need to have ready BEFORE writing a single line of code. Students who skip this section will hit errors in Section 1.

✅ Before you start — complete this checklist:

#	What you need	Where to get it
1	Python 3.11 or higher installed	https://www.python.org/downloads/
2	An AI IDE installed (pick one)	Cursor: https://cursor.sh OR Google Antigravity: https://antigravity.google OR VS Code + GitHub Copilot
3	Git installed	https://git-scm.com/downloads
4	GitHub account	https://github.com
5	OpenAI account + API Key	https://platform.openai.com/api-keys
6	Apify account + API Token	https://apify.com → Settings → Integrations → API Token
7	Supabase account (free tier)	https://supabase.com
8	Streamlit Cloud account	https://streamlit.io/cloud (sign in with GitHub)

🔑 API Keys you will need — keep these ready:

```
OPENAI_API_KEY=sk-...
APIFY_API_TOKEN=apify_api_...
SUPABASE_URL=https://xxxx.supabase.co
SUPABASE_KEY=eyJ...
```

> ⚠️ ***Never share these keys publicly or commit them to GitHub. We will handle this safely in Section 1.***

🗂️ Section 1 — Project Structure

What this prompt does: Creates the complete folder and file structure of the project — exactly matching the reference repo. Every file is created empty at first. This ensures your project looks identical regardless of which AI IDE you are using.

Before running: - Open your AI IDE - Create a new empty folder on your computer called `AI-Job-Recommender` - Open that folder in your AI IDE

📋 Paste this prompt into your AI IDE:

```
Create the following project structure exactly as described. Create all folders and
files listed. Files should be empty for now — we will fill them in later sections.

Root folder: AI-Job-Recommender/

Files in root:
- app.py (empty)
- mcp_server.py (empty)
- __init__.py (empty)
- .python-version (add content: 3.11)
- requirements.txt (add these exact dependencies, one per line:
    streamlit
    openai
    apify-client
    python-dotenv
    supabase
    weasyprint
    pymupdf
    mcp
  )
- packages.txt (add these exact system dependencies, one per line:
    libpango-1.0-0
    libpangoft2-1.0-0
    libglib2.0-0
    libjpeg-dev
    libopenjp2-7-dev
    libffi-dev
    shared-mime-info
  )
- .gitignore (add this exact content:
    # Python virtual environment
    .venv/
    env/
    venv/
    __pycache__/
    *.pyc

    # Environment variables - NEVER commit this
    .env

    # Temporary files
    apify_output.txt
    *.log
    .DS_Store
    *.db
    *_Tailored_*.pdf
  )
- .env (create this file with placeholder values - student will fill in their real
keys:
    OPENAI_API_KEY=your_openai_api_key_here
    APIFY_API_TOKEN=your_apify_token_here
    SUPABASE_URL=your_supabase_url_here
    SUPABASE_KEY=your_supabase_key_here
  )

Subfolder: src/
Files inside src/:
- __init__.py (empty)
- helper.py (empty)
- job_api.py (empty)
- database.py (empty)

After creating all files, show me the complete folder tree so I can verify it looks
correct.
```

✅ Checkpoint — paste this into your AI IDE:

```
Run this in the terminal and show me the output:

    find . -not -path './.venv/*' -not -path './__pycache__/*' | sort

Then verify:
- Root folder contains: app.py, mcp_server.py, __init__.py, .python-version,
requirements.txt, packages.txt, .gitignore, .env
- src/ folder contains: __init__.py, helper.py, job_api.py, database.py

If anything is missing, create it now.
Then install dependencies by running:
    pip install -r requirements.txt

Show me the output of the pip install. If any package fails, fix it.
```

🧠 Section 2 — Creating `helper.py`

What this prompt does: Builds the AI brain of the app. Handles reading PDF resumes, sending prompts to OpenAI GPT, tailoring resumes to job descriptions using a strict agent prompt, and converting HTML to a downloadable PDF using WeasyPrint. Also adds a Mac M1/M2 compatibility fix for WeasyPrint.

Before running: - Open your `.env` file - Replace `your_openai_api_key_here` with your real OpenAI API key:

```
OPENAI_API_KEY=sk-your-real-key-here
```

- Save the `.env` file

📋 Paste this prompt into your AI IDE:

```
Fill in the file src/helper.py with the following complete implementation.

Requirements:
1. Import os, platform, sys at the top
2. Add a Mac M1/M2 WeasyPrint fix: if platform.system() == "Darwin", set
DYLD_LIBRARY_PATH and DYLD_FALLBACK_LIBRARY_PATH to "/opt/homebrew/lib"
3. Import fitz (PyMuPDF), dotenv, openai, apify_client after the platform fix
4. Load .env using load_dotenv(override=True)
5. Read OPENAI_API_KEY and APIFY_API_TOKEN from environment variables
6. Initialize OpenAI client with the API key
7. Initialize ApifyClient with the Apify token

Create these 4 functions:

Function 1: extract_text_from_pdf(uploaded_file)
- Uses fitz (PyMuPDF) to open and extract text from a PDF
- Must handle both Streamlit UploadedFile objects (with .read() method) and direct
file paths
- Returns extracted text as a string

Function 2: ask_openai(prompt, max_tokens=1000)
- Calls gpt-4o-mini model
- Uses temperature 0.5
- System message: "You are a helpful assistant."
- Returns the response content string
- Handles exceptions gracefully

Function 3: tailor_resume(resume_text, job_description, html_template)
- Calls gpt-4o model with temperature 0.3
- System prompt must enforce these STRICT rules:
  * Do not invent roles, projects, metrics, or skills not in the original resume
  * Job titles must remain exactly as they appear in the original resume
  * Do not add certifications or tools not already in the resume
  * All numbers and percentages must be preserved exactly
  * A JD keyword may only be added if the underlying skill already exists in the
resume
- Allowed changes: update tagline, rewrite summary using JD keywords, rephrase
bullets, reorder bullets, surface skills
- HTML template editing rules: fill .name, .tagline, .summary, .competencies with
<span class="tag">, .job blocks, bullets in <ul class="bullets"><li> tags, use <span
class="bold"> for metrics
- Returns the complete tailored HTML string

Function 4: generate_resume_pdf(html_content, output_filename)
- Uses WeasyPrint to convert HTML string to PDF
- Must strip markdown code fences (```html ... ```) if the LLM accidentally returned
them
- Returns True on success, False on failure
- Handles exceptions gracefully
```

✅ Checkpoint — paste this into your AI IDE:

```
Run this test in the terminal:

    python -c "
from src.helper import ask_openai
result = ask_openai('Reply with exactly the words: helper.py is working')
print(result)
"

If it prints a response containing those words, tell me 'Section 2 complete -
helper.py is working'.
If it fails, show me the full error message and fix it automatically before telling me
it is done.
```

🔍 Section 3 — Creating `job_api.py`

What this prompt does: Builds the job hunting engine. Connects to Apify's web scraping platform to fetch live job listings from LinkedIn and Naukri. Also includes a Playwright-based deep scraper for fetching full job descriptions from Naukri when only a snippet is returned.

Before running — 3 manual steps required:

Step 3a — Add your Apify token to `.env`:

```
APIFY_API_TOKEN=apify_api_your_token_here
```

Step 3b — Rent (subscribe to) the LinkedIn Jobs Scraper actor on Apify: 1. Go to https://apify.com and log in 2. Click **Store** in the top navigation 3. Search for: `LinkedIn Jobs Scraper` 4. Find the actor with ID `BHzefUZlZRKWxkTck` (by bebity) 5. Click on it → Click **Try for free** or **Save to My Actors** 6. This subscribes you to the actor so you can call it via API

Step 3c — Rent the Naukri Jobs Scraper actor on Apify: 1. Stay on Apify Store 2. Search for: `Naukri Job Scraper` 3. Find the actor by `muhammetakkurtt` 4. Click **Try for free** or **Save to My Actors**

> 💡 ***Why do we need to rent these actors?*** *Apify actors are pre-built web scrapers created by other developers. When you "rent" or subscribe to one, you get access to run it via the API. The LinkedIn actor (BHzefUZlZRKWxkTck) handles all the complex LinkedIn scraping so we don't have to build it ourselves.*

📋 **Paste this prompt into your AI IDE:**

```
Fill in the file src/job_api.py with the following complete implementation.

Requirements:
1. Import streamlit, os, dotenv, apify_client at the top
2. Load .env using load_dotenv(override=True)
3. Read APIFY_API_TOKEN from environment
4. Initialize ApifyClient with the token
5. Import save_jobs_to_db from src.database (we will create this later - just add the
import)

Create these 3 functions:

Function 1: fetch_linkedin_jobs(search_query, location="india", rows=60)
- Decorated with @st.cache_data(show_spinner=False)
- Calls Apify actor with ID: "BHzefUZlZRKWxkTck"
- run_input must include:
    "title": search_query
    "location": location
    "rows": rows
    "proxy": {"useApifyProxy": True, "apifyProxyGroups": ["RESIDENTIAL"]}
- Calls the actor using
apify_client.actor("BHzefUZlZRKWxkTck").call(run_input=run_input)
- Iterates dataset items:
list(apify_client.dataset(run["defaultDatasetId"]).iterate_items())
- Calls save_jobs_to_db("linkedin", search_query, jobs)
- Returns the jobs list

Function 2: fetch_naukri_jobs(search_query, location="india", rows=60,
deep_scan=False)
- Decorated with @st.cache_data(show_spinner=False)
- Calls Apify actor: "muhammetakkurtt/naukri-job-scraper"
- run_input must include:
    "keyword": search_query
    "maxJobs": rows
- Iterates dataset items same as above
- Calls save_jobs_to_db("naukri", search_query, jobs)
- If deep_scan=True: call fetch_full_details_batched on the top 10 job URLs (field:
jdURL), map results back to jobs by URL, re-save to db
- Returns the jobs list

Function 3: fetch_full_details_batched(urls)
- NOT cached (no decorator)
- If urls list is empty, return []
- Calls Apify actor: "apify/playwright-scraper"
- run_input must include:
    "startUrls": [{"url": url} for url in urls]
    "pageFunction": a JavaScript async function that:
        - Waits for selector '.jd-description, .job-desc, [class*="job-desc"]' with
10s timeout
        - Evaluates the page to get innerText of the first matching element
        - Returns {url: context.request.url, full_description: the text trimmed}
    "proxyConfiguration": {"useApifyProxy": True, "apifyProxyGroups": ["RESIDENTIAL"]}
    "maxConcurrency": 5
- Returns list of results from the dataset
```

✅ Checkpoint — paste this into your AI IDE:

```
Run this test in the terminal:

    python -c "
import os
from dotenv import load_dotenv
load_dotenv()
from apify_client import ApifyClient
client = ApifyClient(os.getenv('APIFY_API_TOKEN'))
user = client.user('me').get()
print('Apify connected. Username:', user.get('username'))
"

If it prints your Apify username, tell me 'Section 3 complete - Apify connection
working'.
If it fails, show me the full error and fix it.
Do NOT run an actual job scrape yet - that costs Apify credits.
```

🖥️ Section 4 — Creating `app.py`

What this prompt does: Builds the complete Streamlit web application — the entire user interface. Includes login/logout, PDF upload, resume summary, skill gap analysis, career roadmap, job search trigger, LinkedIn job cards, Naukri job cards, and the resume tailoring + PDF download flow. This is the main file users interact with.

Before running: - Sections 2 and 3 must be complete and checkpoints passed - Make sure your `.env` has both `OPENAI_API_KEY` and `APIFY_API_TOKEN` filled in - We will NOT integrate the database in this section yet — that comes in Section 5

📋 Paste this prompt into your AI IDE:

```
Fill in app.py with a complete Streamlit application for an AI Job Recommender System.

Page config: title "AI Job Recommender - Login", icon "💼"

Authentication system:
- Use st.session_state["authenticated"] to track login state
- Show a login page first if not authenticated
- Login form with email and password fields
- Accept only: email = "demo@nomail.com", password = "password"
- On wrong credentials show error: "❌ Invalid Email or Password"
- On success set authenticated = True and st.rerun()

Sidebar (shown after login):
- Show "Logged in as: demo@nomail.com"
- Logout button that sets authenticated = False and reruns
- Section "📊 Database Stats" that tries to import supabase from src.database and show
LinkedIn and Naukri job counts - wrap in try/except, show "Stats unavailable" on error

Main page:
- Title: "💼 AI Job Recommender System"
- Subtitle: "Upload your resume and get recommendations based on your job skills and
experience from LinkedIn and Naukri."

Custom CSS for job cards:
- Class .job-card: white background, border-radius 16px, padding 24px, border 1px
solid #e1e4e8, subtle box shadow
- Class .linkedin-card: border-left 5px solid #0077b5
- Class .naukri-card: border-left 5px solid #ff751a
- Include styles for: .job-title (20px bold), .company-info, .meta-
row, .badge, .badge-easy (blue), .badge-salary (green), .description-preview (2-line
clamp), .apply-btn (blue button style), .view-link, .avatar

Resume upload section:
- st.file_uploader for PDF files only
- On upload: extract text using extract_text_from_pdf from src.helper, store in
session_state
- Generate resume summary using ask_openai - store in session_state
- Extract candidate name using ask_openai - store in session_state

After upload show these sections with st.markdown styled divs:
1. "📄 Resume Summary" - show the AI summary in a styled gray box
2. "🛠️ Skill Gaps & Missing Areas" - ask GPT for skill gaps and certifications needed
3. "🚀 Future Roadmap & Preparation Strategy" - ask GPT for career roadmap
4. Show "✅ Analysis Completed Successfully!" after all three

Job Recommendations button labeled "🧲 Get Job Recommendations 🧲":
- Use ask_openai to extract the current designation from resume summary
- Use ask_openai to get comma-separated job keywords from resume summary
- Build linkedin_query = designation only
- Build naukri_query = designation + top 3 keywords (excluding designation itself)
- Store both queries in session_state
- Call fetch_linkedin_jobs(linkedin_query, rows=60) from src.job_api
- Call fetch_naukri_jobs(naukri_query, rows=60) from src.job_api
- Store results in session_state

LinkedIn job cards (shown when linkedin_jobs exists in session_state):
- Show header with search query used
- For each job render an HTML job card with: job title, company name (with avatar
showing first letter), location, posted time, salary, Easy Apply badge if
easyApply=True or applyType=="EASY_APPLY", 200-char description preview
- Add expander "Tailor Resume for this Job" with button "✨ Create Customised Resume":
  * Load resume_template.html from same directory as app.py
  * Call tailor_resume(resume_text, full_desc, html_template)
  * Call generate_resume_pdf(tailored_html, pdf_filename)
```

```
  * Show download button for the PDF
- Add expander "Read More Details" showing full job description

Naukri job cards (shown when naukri_jobs exists in session_state):
- Similar card layout with orange left border
- Show rating badge if ambitionBoxData.AggregateRating exists
- Add expander "Tailor Resume for this Job":
  * If job description is short (under 500 chars), first call
fetch_full_details_batched([job_url]) to get full JD
  * Then tailor and generate PDF same as LinkedIn flow
- Add expander "Read More / Fetch Full Details":
  * If description is short, show snippet and a button "🔍 Fetch 100% Full
Description" that calls fetch_full_details_batched and reruns
  * If full description exists, show it in a styled box

All AI calls must use session_state to avoid re-running on every Streamlit rerender.
Include a clean_html(raw_html) helper function that strips HTML tags using regex.
Import APP_DIR = os.path.dirname(os.path.abspath(__file__)) for reliable file path
resolution.
```

✅ Checkpoint — paste this into your AI IDE:

```
Run the Streamlit app in the terminal:

    streamlit run app.py

Then:
1. Open the URL shown (usually http://localhost:8501)
2. Log in with email: demo@nomail.com and password: password
3. Confirm you can see the main dashboard with the upload button

Tell me what you see on screen.
If the app crashes or shows an error, paste the full terminal error here and fix it
automatically.

Note: We are NOT testing job fetching yet. We are only confirming the app launches and
login works.
```

🗄️ Section 5 — Creating `database.py` with SQLite

What this prompt does: Adds local data persistence. Every time jobs are fetched from Apify, they get saved to a local SQLite database file on your computer. This means jobs are cached and don't need to be re-fetched every time you reload the app. This section builds the SQLite layer only — Supabase cloud storage comes in Section 5.2.

Before running: - Sections 2, 3, and 4 must be complete - No new accounts or keys needed for SQLite — it is built into Python

---

📋 Paste this prompt into your AI IDE:

```
Fill in src/database.py with a SQLite-based persistence layer.

Requirements:
1. Import os, json, datetime, dotenv, and sqlite3
2. Load .env using load_dotenv(override=True)
3. Set DB_PATH to a file called "jobs_repository.db" in the project root (use
os.path.dirname(os.path.dirname(os.path.abspath(__file__))) to get the root)
4. Set supabase = None for now (we will add Supabase in Section 5.2)

Create these functions:

Function 1: init_db()
- Connects to SQLite at DB_PATH
- Creates table linkedin_jobs_v2 if not exists with columns:
    job_id TEXT PRIMARY KEY, title TEXT, company TEXT, location TEXT,
    salary TEXT, posted_at TEXT, job_description TEXT, search_query TEXT,
    fetched_at DATETIME, raw_data TEXT
- Creates table naukri_jobs_v2 if not exists with same columns
- Commits and closes connection

Function 2: save_jobs_to_db(source, search_query, jobs_list)
- source is either "linkedin" or "naukri" - use this to pick the right table name
(linkedin_jobs_v2 or naukri_jobs_v2)
- timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
- For each job in jobs_list:
    * job_id = str(job.get('jobId') or job.get('id') or job.get('url') or
job.get('jdURL'))
    * Skip if job_id is None or "None"
    * Build a record dict with all columns
    * raw_data = json.dumps(job)
- Save to SQLite using INSERT OR REPLACE
- Handle exceptions gracefully with print statements

Function 3: get_all_keys(source)
- Utility function that reads raw_data from the first 10 rows of the relevant table
- Returns a sorted list of all unique keys found across those records
- Useful for debugging what fields Apify is returning

Call init_db() at the bottom of the file so the tables are created on import.
```

✅ Checkpoint — paste this into your AI IDE:

```
Run this test in the terminal:

    python -c "
from src.database import init_db, save_jobs_to_db
init_db()

# Save a fake test job
test_job = {
    'jobId': 'test_001',
    'title': 'AI Product Manager',
    'companyName': 'Test Corp',
    'location': 'Delhi, India',
    'jobDescription': 'This is a test job description.'
}
save_jobs_to_db('linkedin', 'AI Product Manager', [test_job])
print('database.py is working - test job saved successfully')

# Verify it was saved
import sqlite3, os
db_path = os.path.join(os.path.dirname(os.path.abspath('.')), 'jobs_repository.db')
# Try local path too
import glob
db_files = glob.glob('**/*.db', recursive=True) + glob.glob('*.db')
print('Database files found:', db_files)
"

If it prints 'database.py is working', tell me 'Section 5 complete'.
If it fails, show me the error and fix it.
```

🎭 Section 5.1 — Playwright Scraper for Full Naukri JDs

What this prompt does: Solves a specific problem: Naukri's job listings page is a JavaScript-heavy React app. When Apify's basic scraper visits it, it only gets a short snippet of the job description — not the full text. This section explains why that happens and adds a Playwright-based deep scraper that launches a real browser to render the page and extract the full JD.

The problem — why Naukri gives us a snippet:

> *Naukri.com loads job descriptions dynamically using JavaScript after the page loads. A basic HTTP scraper sees the HTML before JavaScript runs, so it only captures what's in the initial HTML — usually just a 2-3 line snippet. To get the full JD, you need a real browser that can wait for JavaScript to finish rendering.*

The solution — Playwright:

> *Apify's* `playwright-scraper` *actor launches a real Chromium browser, waits for the page to fully render, then extracts the complete job description. This is already included in* `fetch_full_details_batched()` *from Section 3. This section makes the UI in* `app.py` *smart enough to detect when a JD is a snippet and trigger the full fetch automatically.*

Before running: - Section 3 and Section 5 must be complete - No new accounts needed — this uses your existing Apify token

---

📋 Paste this prompt into your AI IDE:

```
Update src/job_api.py to ensure fetch_full_details_batched works correctly with these
exact settings.

The pageFunction JavaScript inside the run_input must be exactly this:

async function pageFunction(context) {
    const { page } = context;
    await page.waitForSelector('.jd-description, .job-desc, [class*="job-desc"]',
{ timeout: 10000 }).catch(() => {});
    const full_jd = await page.evaluate(() => {
        const el = document.querySelector('.jd-description') ||
                   document.querySelector('.job-desc') ||
                   document.querySelector('[class*="job-desc"]');
        return el ? el.innerText : "";
    });
    return {
        url: context.request.url,
        full_description: full_jd.trim()
    };
}

Also update app.py so that in the Naukri job cards section:
- A job description is considered a "snippet" if it is less than 500 characters
- When the user clicks "Tailor Resume" on a snippet job: first call
fetch_full_details_batched([job_url]) to get the full JD, update the job object with
the full description, then proceed with tailoring
- When the user clicks "Fetch Full Description" in the expander: call
fetch_full_details_batched([job_url]), update the job and re-save to database, then
call st.rerun()
- Show "⚠️ Current description is a snippet." label when description is short
- Show "✅ Full Description Fetched" label when full description is available
```

✅ Checkpoint — paste this into your AI IDE:

```
Run this test in the terminal:

    python -c "
from src.job_api import fetch_full_details_batched

# Test with a real Naukri URL (use a job URL you can find on naukri.com)
# We will use a known stable URL for testing
test_urls = ['https://www.naukri.com/job-listings-product-manager-bangalore-0-to-3-
years-020325009173']

print('Testing Playwright scraper...')
print('Note: This will consume Apify credits. Press Ctrl+C to cancel if you want to
skip.')
results = fetch_full_details_batched(test_urls)
if results:
    print('Playwright scraper working. Description length:',
len(results[0].get('full_description', '')))
    print('Section 5.1 complete')
else:
    print('No results returned - check Apify actor subscription')
"

If it returns results, tell me 'Section 5.1 complete - Playwright scraper working'.
If it fails with an actor not found error, remind me to subscribe to the playwright-
scraper actor on Apify Store.
If it fails with any other error, show the full error and fix it.
```

☁️ Section 5.2 — Adding Supabase Cloud Storage

What this prompt does: Upgrades the database layer from local-only SQLite to cloud-first Supabase with SQLite as a fallback. Jobs saved to Supabase are accessible from anywhere — including when the app is deployed on Streamlit Cloud. The app tries Supabase first, and if it fails (or isn't configured), it falls back to SQLite silently.

Before running — Supabase setup (4 manual steps):

Step 5.2a — Create a Supabase project: 1. Go to https://supabase.com and log in 2. Click **New Project** 3. Name it `ai-job-recommender` 4. Choose a region close to you 5. Set a database password and save it

Step 5.2b — Create the two tables in Supabase: 1. In your Supabase project, click **SQL Editor** 2. Run this SQL:

```
CREATE TABLE IF NOT EXISTS linkedin_jobs_v2 (
    job_id TEXT PRIMARY KEY,
    title TEXT,
    company TEXT,
    location TEXT,
    salary TEXT,
    posted_at TEXT,
    job_description TEXT,
    search_query TEXT,
    fetched_at TIMESTAMP,
    raw_data TEXT
);

CREATE TABLE IF NOT EXISTS naukri_jobs_v2 (
    job_id TEXT PRIMARY KEY,
    title TEXT,
    company TEXT,
    location TEXT,
    salary TEXT,
    posted_at TEXT,
    job_description TEXT,
    search_query TEXT,
    fetched_at TIMESTAMP,
    raw_data TEXT
);
```

Step 5.2c — Get your Supabase credentials: 1. Go to **Project Settings** → **API** 2. Copy the **Project URL** → this is your `SUPABASE_URL` 3. Copy the **anon public** key → this is your `SUPABASE_KEY`

Step 5.2d — Add to your `.env` file:

```
SUPABASE_URL=https://xxxx.supabase.co
SUPABASE_KEY=eyJhbGc...
```

📋 Paste this prompt into your AI IDE:

```
Update src/database.py to add Supabase as the primary storage with SQLite as fallback.

Changes required:

1. Add imports: from supabase import create_client, Client

2. Read SUPABASE_URL and SUPABASE_KEY from environment variables

3. Initialize Supabase client:
    supabase: Client = None
    if SUPABASE_URL and SUPABASE_KEY:
        try:
            supabase = create_client(SUPABASE_URL, SUPABASE_KEY)
        except Exception as e:
            print(f"Failed to connect to Supabase: {e}")

4. Update save_jobs_to_db() to try Supabase FIRST, then SQLite as fallback:
    - If supabase client exists: loop through records and call
supabase.table(table_name).upsert(record).execute()
    - Print success message with count
    - If Supabase raises an exception: print the error and fall through to SQLite
    - Always also save to SQLite (as backup) regardless of Supabase result

5. Keep init_db(), get_all_keys() exactly as they are from Section 5.

Keep the init_db() call at the bottom of the file.
```

✅ Checkpoint — paste this into your AI IDE:

```
Run this test in the terminal:

    python -c "
from src.database import supabase, save_jobs_to_db

if supabase:
    print('Supabase client initialized successfully')

    # Save a test job to Supabase
    test_job = {
        'jobId': 'supabase_test_001',
        'title': 'AI Product Manager',
        'companyName': 'Supabase Test Corp',
        'location': 'Delhi, India',
        'jobDescription': 'Testing Supabase integration.'
    }
    save_jobs_to_db('linkedin', 'AI PM test', [test_job])
    print('Test job saved to Supabase')

    # Verify by reading back
    result = supabase.table('linkedin_jobs_v2').select('job_id, title').eq('job_id',
'supabase_test_001').execute()
    print('Read back from Supabase:', result.data)
    print('Section 5.2 complete - Supabase working')
else:
    print('Supabase not connected - check your SUPABASE_URL and SUPABASE_KEY in .env')
"

If it prints 'Section 5.2 complete', tell me we are ready for Section 6.
If Supabase is not connected, walk me through what might be wrong with the
credentials.
```

🐙 Section 6 — Push to GitHub

What this prompt does: Creates a new GitHub repository for the student's project and pushes all the code. This is required before deploying to Streamlit Cloud in Section 7.

Before running: - Go to https://github.com and log in - Create a **new repository** called `AI-Job-Recommender` - Set it to **Public** - Do NOT initialize with README (leave it empty) - Copy the repository URL (e.g. `https://github.com/yourusername/AI-Job-Recommender.git`)

📋 Paste this prompt into your AI IDE:

```
Run these git commands in the terminal one by one. Replace YOUR_GITHUB_URL with your
actual GitHub repository URL.

First, initialize git and make the first commit:

    git init
    git add .
    git status

Show me the output of git status. Confirm that .env is NOT listed in the files to be
committed (it should be ignored by .gitignore).

If .env is listed, stop immediately and do not commit. Fix the .gitignore first.

If .env is not listed, proceed with:

    git commit -m "Initial commit: AI Job Recommender App"
    git branch -M main
    git remote add origin YOUR_GITHUB_URL
    git push -u origin main

Show me the output after each command.
```

✅ Checkpoint — paste this into your AI IDE:

```
Run this in the terminal:

    git log --oneline
    git remote -v

Confirm:
1. There is at least one commit showing
2. The remote origin points to your GitHub URL

Then open your GitHub repository URL in the browser and confirm all files are visible
EXCEPT .env.

Tell me:
- How many files are visible in your GitHub repo?
- Is .env visible? (it must NOT be)

If .env is visible on GitHub, we have a security problem and must fix it immediately
before continuing.
```

🚀 Section 7 — Deploy on Streamlit Cloud

What this prompt does: Deploys the app to the internet using Streamlit Cloud — a free hosting service for Streamlit apps. After this section, your app will have a public URL that anyone can access.

Before running — 2 manual steps:

Step 7a — Add secrets to Streamlit Cloud: After connecting your repo (step below), you will add your API keys as Streamlit Secrets — this replaces the `.env` file in production.

Step 7b — Connect GitHub to Streamlit Cloud: 1. Go to https://streamlit.io/cloud 2. Sign in with your GitHub account 3. Click **New app** 4. Select your `AI-Job-Recommender` repository 5. Set branch to `main` 6. Set Main file path to `app.py` 7. Click **Advanced settings** → **Secrets** 8. Add your secrets in TOML format:

```
OPENAI_API_KEY = "sk-your-key-here"
APIFY_API_TOKEN = "apify_api_your-token-here"
SUPABASE_URL = "https://xxxx.supabase.co"
SUPABASE_KEY = "eyJ..."
```

1. Click **Deploy**

📋 Paste this prompt into your AI IDE:

```
The app is being deployed on Streamlit Cloud. Streamlit Cloud reads secrets from
st.secrets instead of from a .env file.

Update app.py and all src/ files to be compatible with Streamlit Cloud deployment:

1. In src/helper.py: when loading OPENAI_API_KEY, try os.getenv() first, then fall
back to st.secrets if available. Same for APIFY_API_TOKEN.

2. In src/database.py: when loading SUPABASE_URL and SUPABASE_KEY, try os.getenv()
first, then fall back to st.secrets if available.

3. In src/job_api.py: when loading APIFY_API_TOKEN, try os.getenv() first, then fall
back to st.secrets if available.

The pattern for each should be:
    import streamlit as st
    value = os.getenv("KEY_NAME")
    if not value:
        try:
            value = st.secrets.get("KEY_NAME")
        except:
            pass

After making these changes, commit and push to GitHub:
    git add .
    git commit -m "Add Streamlit Cloud secrets compatibility"
    git push
```

✅ Checkpoint — paste this into your AI IDE:

```
After Streamlit Cloud finishes deploying (usually 2-3 minutes):

1. Open the public URL Streamlit gave you
2. Try logging in with demo@nomail.com / password
3. Upload a PDF resume
4. Check if the resume summary appears

Tell me the public URL of your deployed app.

If the deployment fails with an error:
- Go to Streamlit Cloud → your app → Logs
- Copy the error message here
- I will fix it and we will push the fix to GitHub
```

🔐 Section 8 — Login Credentials

What this prompt does: The app currently has hardcoded credentials (`demo@nomail.com` / `password`). This section explains what that means, why it is there, and how to change it to your own email and password so only you (or your students) can access the deployed app.

Before running: - Think of the email and password you want to use - This is a simple authentication gate — suitable for demos and teaching purposes - For a production app with real users, you would use a proper auth system like Supabase Auth

---

📋 Paste this prompt into your AI IDE:

```
Update the login system in app.py.

Current hardcoded credentials:
    email = "demo@nomail.com"
    password = "password"

Make these changes:

1. Move the credentials to the .env file and Streamlit secrets:
    Add to .env:
        APP_EMAIL=your_chosen_email@example.com
        APP_PASSWORD=your_chosen_password

2. In app.py, read the credentials from environment:
    import os
    APP_EMAIL = os.getenv("APP_EMAIL", "demo@nomail.com")
    APP_PASSWORD = os.getenv("APP_PASSWORD", "password")

3. Also try st.secrets as fallback (same pattern as Section 7):
    if not APP_EMAIL:
        try: APP_EMAIL = st.secrets.get("APP_EMAIL")
        except: pass

4. Update the login check to use these variables:
    if email == APP_EMAIL and password == APP_PASSWORD:

5. Update the sidebar to show the logged-in email dynamically:
    st.write(f"Logged in as: **{APP_EMAIL}**")

6. Add APP_EMAIL and APP_PASSWORD to Streamlit Cloud secrets (remind the user to do
this manually in the Streamlit Cloud dashboard).

7. Commit and push:
    git add .
    git commit -m "Move login credentials to environment variables"
    git push
```

✅ Final Checkpoint — paste this into your AI IDE:

```
Do a complete end-to-end test of the deployed app:

1. Open the Streamlit Cloud URL
2. Log in with your new credentials
3. Upload a PDF resume (use any PDF - even a simple one)
4. Confirm resume summary appears
5. Click "Get Job Recommendations"
6. Confirm LinkedIn and Naukri jobs appear
7. Click "Tailor Resume" on any job
8. Confirm a tailored PDF downloads successfully

Report what works and what does not. Fix any errors you find.

Then run a final check locally:
    git log --oneline | head -5

Show me the last 5 commits. This confirms the full build history is on GitHub.

Congratulations - your AI Job Recommender App is live! 🎉
```

📁 Final Project Structure

When everything is complete, your project should look exactly like this:

```
AI-Job-Recommender/
│
├── app.py                          ← Streamlit web app (main entry point)
├── mcp_server.py                   ← FastMCP server for AI agents
├── __init__.py                     ← Package marker
├── resume_template.html            ← HTML template for tailored resumes
├── .python-version                 ← Python 3.11
├── requirements.txt                ← Python dependencies
├── packages.txt                    ← System dependencies (for Streamlit Cloud)
├── .gitignore                      ← Excludes .env, __pycache__, *.db
├── .env                            ← Your secrets (NEVER committed to GitHub)
│
└── src/
    ├── __init__.py                 ← Makes src a Python package
    ├── helper.py                   ← PDF reader + GPT + WeasyPrint
    ├── job_api.py                  ← Apify LinkedIn + Naukri scrapers
    └── database.py                 ← Supabase + SQLite persistence
```

🔑 Quick Reference — All API Keys & Where to Get Them

Key	Where to get it	Used in
`OPENAI_API_KEY`	https://platform.openai.com/api-keys	`helper.py`
`APIFY_API_TOKEN`	https://apify.com → Settings → Integrations	`job_api.py`
`SUPABASE_URL`	Supabase → Project Settings → API	`database.py`
`SUPABASE_KEY`	Supabase → Project Settings → API (anon public)	`database.py`
`APP_EMAIL`	You choose this	`app.py`
`APP_PASSWORD`	You choose this	`app.py`

🎓 Reference Repo

You can compare your final code with the original repo at any time: 👉 https://github.com/ashumishra2104/Building_MCP_Server_and_Client/tree/main/Job_Recommender_App

---

Guide created for AI Product Management students. Built with Google Antigravity.